Uncertain reflections

Clayton Lewis
University of Colorado Boulder

Much of my career was motivated by a seemingly clear idea that now seems questionable: programming should be easier. In 1968, when I started thinking about this, the pain points were many, and obvious. Programming required mastering many concepts that were outside your domain of interest, like floating point numbers, pointers, and so on (see Note 1). It was hard to deploy these concepts, even if you understood them, again from the point of view of wanting to focus on one's target problem(s). It was hard to understand what programs would do, once written.

The obvious attack on these problems was to try to build programs out of different stuff, that would (one hoped) be easier to understand and work with. Approaches that my students and I worked on included generalization of the spreadsheet, pictorial rewrite rules, and visual and nonvisual dataflow computing, as well as flirtation with logic programming. We didn't think in terms of substrates, though it would have made sense to do so, and some limitations of our (largely ineffectual) research program might have been addressed had we done so (see Note 2.)

Why does this now seem questionable? I now find that LLM-based coding tools are bypassing many of these pain points. In a paper for a workshop at <programming 22> (Lewis, 2022) I showed that Codex (as it then was) made it possible to write a sample program from the Boxer educational programming project while knowing and understanding almost nothing about the code.This year, using Claude, I found that I could get a working program with a single prompt, while knowing nothing about the code, full stop (see Note 3).

So, many of the aims that motivated me have been achieved. While programming can still be difficult, in many cases the difficulties come not from programming, and its conceptual baggage, but from the problem domain one is working in. I never had the ambition to solve those difficulties (but see below.)

Is there anything left for people interested in substrates to do? The uncertainty in the title comes in here. It is uncertain where the trajectory of AI coding tools is going to take us. The relatively trivial programs that I focussed on in my programming research can now be solved, without touching any of the gnarly aspects of coding at all. But how much "real programming" can be handled?

Many well-informed people think, all of it. My own reflections on human cognition, and its relationship to AI (see Lewis, 2025) make me think there is no reason why that won't be true. It's already apparent that AI tools have enormous advantages over people-- GPT knows far more languages and frameworks than any human, for example. They can process huge codebases in bulk in ways that people can't match. So far, their ability to solve really hard problems isn't at top human level, but what's the argument that they can't be? I don't see one. But I'm not confident about that.

I think It's plausible, though, that fairly soon, nobody will need to know how to code, in order to get programs. This puts one of my favorite papers on the dustheap of history. It was an argument for human-centered language design, using qualitative methods (Lewis, 2017). But who cares, if we humans can get programs without coding?

There may still be reasons to understand code, even so. One reason has been there all along: it's not enough to have a program: often one needs to know that the program works. Differences in the stuff of programming may make a difference there.

Can't the task of knowing that a program works be delegated to the AI tools? I'd argue, not entirely. Knowing what behavior is correct depends on understanding the program's behavior in terms of the target problem domain. Informal understanding of problem domains is quite unreliable. It appears that some kind of formalization is needed to make it more reliable, and that's an aspect of programming, or is at least programming adjacent (but see Note 4.)

That challenge, of understanding what correct behavior is, connects to another reason to understand something about programming. Jeanette Wing's influential computational thinking manifesto (2006) overstated the case (IMO), but there are situations in which thinking about computation can suggest useful ways to think about problems that aren't (superficially) computational. Turing's work in mathematics is an example. If nobody understands code, nobody will be able to think those things, or at least, not without reinventing a lot of structure (Turing of course had to invent it, not reinvent it.)

Andy diSessa's Boxer project is an extended argument of just this kind. Physics isn't superficially computational, but Andy argues that being able to represent physics computationally has conceptual value.

What research agenda emerges from these reflections? I've sketched two things to think about. What conceptual tools can help people define correctness in the problem domains they care about? What aspects of computation provide intellectual value to those who understand them, leaving aside the value of the ideas for creating programs? Are there ways to embody these aspects in simple, easy to understand forms, so that we plodding meat machines can readily profit from them?

Notes

Note 1. For many people, computation itself became the target domain of interest. That has led to a lot of problems, as Curtis, Krasner and Iscoe documented (1988). Many software developers thought they didn't need to understand the target domain, since they could code. The "thin spread of application domain knowledge" was and is a huge source of waste.

Note 2. Our research program never got beyond proof of concept demonstrations. These suffered from the walled garden problem, where the ideas existed in an isolated computational ecosystem. Little real work can get done that way, and substrates work should recognize that.

Note 3. Here's the prompt I used:

> write a Web app, packaged as one file, that displays a 3x3 grid of coins, and has buttons that flip all the coins in any row, column, or diagonal. Also arrange that clicking on any coin flips that coin. Make the coins all heads to start with.

The request, "packaged as one file", does reflect experience with other models not producing an easy to use product. I didn't check that that was needed for Claude. Anyway, if one objects to that level of knowledge about code that could be stuck in a prompt prefix (for example for children.)

Note 4. I've written this as if "the target problem domain" is the exclusive province of the human user, but that's not true. Ai systems already "know" a great deal about many target problem domains, and will likely play a large role in defining correct behaviors. But how will they communicate their understanding to people? Some language to help articulate correctness seems necessary.

References

Curtis, B., Krasner, H., & Iscoe, N. (1988). A field study of the software design process for large systems. *Communications of the ACM*, 31(11), 1268-1287.

DiSessa, A. A. (2018). Computational literacy and “the big picture” concerning computers in mathematics education. *Mathematical thinking and learning*, 20(1), 3-31.

Lewis, C. (2017) Methods in user oriented design of programming languages. In *Proc. PPIG 2017 Psychology of Programming Annual Conference*, Delft, Netherlands, 1-3 July 2017. <https://www.ppig.org/files/2017-PPIG-28th-lewis.pdf >

Lewis, C. (2022, March). Automatic Programming and Education. In *Companion Proceedings of the 6th International Conference on the Art, Science, and Engineering of Programming* (pp. 70-80). <https://dl.acm.org/doi/pdf/10.1145/3532512.3539664 >

Lewis, C. (2025) *Artificial Psychology: Learning from the Unexpected Capabilities of Large Language Models*. Springer Nature.

Wing, J. M. (2006). Computational thinking. *Communications of the ACM*, 49(3), 33-35.

What is (and isn’t) a substrate?
- What values guide our research?
- What are the potential benefits of substrates?
- What can we learn from past attempts?
- What are the research problems?
- What do we disagree about?
- What research methodologies should we use?
- What I’d like to see come out of this workshop
- Details of my personal research vision

********** PAPER 2 **********
AUTHORS: Clayton Lewis, Clayton Lewis and Clayton Lewis
TITLE: Unconfident reflections

+++++++++ REVIEW 1 (Gilad Bracha) +++++++++
I like this paper because it grapples with the elephant in the substrates room: does AI make all our work obsolete? The author admits he doesn't know - which is also great, and a lesson most of humanity could take to heart.

While most of the CS/PL community is in denial, I tend to agree that conventional programming is on its way out. Programs, however, are here to stay. We need programs because they have precise, verifiable meanings and because running them is much more efficient than having an LLM try and solve the same problem directly. Thus, the role of AI is not to take direct action (as agentic AI promises) which is unreliable/dangerous as well as inefficient, but to write programs to take action. The programs can be verified and compiled.

The paper alludes to verification and I am in agreement. I also agree that substrates are for constructing and testing high level specifications rather than conventional programming (at leats, I agree if that is what the paper is saying, as I think it is). These specifications may be be tested by observation; or they may be tests or even formal specifications at the level of theorem provers. Most of all, I agree that all this may be wishful thinking on our part, because we love substrates and cannot face up to the fact that they may be irrelevant.

+++++++++ REVIEW 2 (Jonathan Edwards) +++++++++
One-line summary: AI solves end-user programming; how do we supervise it? Can we learn from it?

The opening hits hard: "Much of my career was motivated by a seemingly clear idea that now seems questionable: programming should be easier." Clayton sounded the alarm early, and others I equally respect are repeating it. I will indulge myself by using this review to try to put my skepticism into words.

I think we should distinguish two claims: 1) AI will improve the productivity of skilled programmers; and 2) AI will de-skill programming so end users can build their own software. Claim 1 is clearly true already, the only question is to what extent. But claim 2 is much stronger and the basis for the more extreme predictions.

I am skeptical of claims by experts that AI can program because these experts are comfortable fixing the AI's mistakes. The Curse of Knowledge is that we can't perceive the difficulties non-experts experience. Can AI clone a median SaaS app and support it in production, solving errors and adding new features, all without any experts being involved? I haven't seen a credible attempt to test that.

+++++++++ REVIEW 3 (Antranig Basman) +++++++++
This response is taken from a blog posting https://ponder.org.uk/post/2025-04-19-new-notation-era/ that I wrote largely in response to Clayton’s submission:

I argue that there has never been a more promising time for developing fruitful new notations.

Some, considering the enormous success of modern LLMs in solving increasingly complex “problems” – in terms of translating a verbal or pictorial design specification into code – means programming is now a “solved” problem. They imagine a future where humans no longer need to understand or write code.

I believe this is wrong on several accounts. The central account is ignoring the extent to which coding is a *social activity*, that is, one that establishes long term relationships amongst communities of use around shared artefacts. Even in the purest of disciplines, mathematics, it has long been recognised, for example in Lakatos’ 1959-1961

paper [What does a mathematical proof prove?](https://sidoli.w.waseda.jp/Lakatos_1978.pdf) that the primary function of a mathematical proof, rather than to constitute a formal specification of a chain of reasoning, is to convince the human reader that its result is true. In doing so it establishes a dialogue amongst related communities, which can only happen in terms of their recognition of shared distinctions that are of interest to them – for example, polyhedra, or functions. One characterisation of the roles of these distinctions is Star and Griesamer's 1989 notion of [Boundary Objects](https://en.wikipedia.org/wiki/Boundary_object).

Rather than representing a **solution to a problem**, most deployed codebases represent a **locus of discussion**. An example of such a locus is the thread on this [MapLibre GitHub issue](https://github.com/maplibre/maplibre-gl-js/issues/793) where contributors debate how to distinguish event sources in an open-source mapping library. This discussion relates the concerns of those using a system to particular structures in its implementation – the discussants need to refer to parts of the library, long in maintenance, in terms which are recognisable to each other as stable landmarks.

Code that can be signed off as a one-off solution to a closed form programming puzzle or scripting task is an important constituency but not a dominant one.

Here are two reasons why the rise of LLMs makes the development of new notations more promising and rewarding, rather than less.

Oversight of code

The proliferation of LLM authorship of code is indeed accelerating and unstoppable. This puts an increasing burden on humans to review and ensure that such code aligns with a community's goals. There have been no shortage of rather quaint posts arguing that [writing efficient code is a moral good](https://tomgamon.com/posts/is-it-morally-wrong-to-write-inefficient-code/) because of its reduced energy consumption. We are now at the point that the energy expended at runtime by typical code during its deployed lifetime is totally dwarfed by that consumed by AI during its composition. Any notation which eases the burden of oversight of AI – that is, makes it as obvious as possible whether the code it has produced is aligned with the goals of its community – is going to offer massive savings. Saving one round trip to an AI based on a notational niggle or shortening its "chain of thought" on a task would be a great result.

Even a wholly vibe coded app with no human community around its expression could benefit from this.

I argue that an important way to ease the path to designs which are **obviously correct** to a human or AI overseer is to minimise their [divergence](https://ponder.org.uk/term/divergence) – any discrepancy between their runtime state and the state from which they can be authored. I and others in the [substrates](https://ponder.org.uk/term/substrate) community are working on notations to achieve this.

Decline of incumbency

At the same time as increasing the stakes for better notations, it has never been easier to deploy them. Typically a framework or language faces a huge incumbency barrier – the years of costs sunk into a particular system make it prohibitive to move to alternatives even if the advantages are huge. However, translation tasks are amongst the easiest for today's LLMs – these are highly constrained tasks with clear metrics for success. Anthropic's [documentation for Claude Code](https://www.anthropic.com/engineering/claude-code-best-practices) describes letting loose an agentic coder on the task of translating 2000 source files from React to Vue (doubtless a very popular industry activity at present). Code today has never been less tied to incumbent notations.

I predict that 30 years from now, those predicting the end of human readership of programming notations will seem as misguided as Richard Gabriel's 1993 essay [The End of History and the Last Programming Language](https://www.dreamsongs.com/Files/PatternsOfSoftware.pdf) which concludes that "the last programming language is C".

++++++++++ REVIEW 4 (Tom Larkworthy) +++++++++
The author calls out the elephant in the room: the original research topic programming accessibility has been technically solved by LLMs. The author suggest the problem has shifted, how we know if the problem has been solved, which is sort of program specification but we have NLP and multi-modal reasoning now.
I would also add that I think maintaining these AI developed system will need new strategies as well.
The AI field is developing new substrates e.g. OpenAI Canvas, Clause Artifacts. Should we be researching how these tools work? Should we be exploring the frontier? There are many directions to go but the pace of innovation is difficult to know where to invest.
The author thinks AI will end some existing substrate research threads (look at how AI ripped through computational linguistics), and creates new. I think this will be an important topic to discuss in the workshop, is programming a dead end?